Exp:7

Aim: Artificial Neural Network (ANN) for Customer Churn Prediction

1. Dataset Source

The dataset used in this experiment is:

- Churn Modelling Dataset
- Source Link: https://www.kaggle.com/datasets/shrutimechlearn/churn-modelling

2. Dataset Description

The dataset contains bank customer information used to predict whether a customer will leave the bank (churn) or not.

🔹 **Key Characteristics:**

- Total Records: **10,000**
- Features: **14 columns**
- Target Variable: **Exited (0 or 1)**

🔹 **Feature Description:**

Feature	Description
RowNumber	Row index (removed)
CustomerId	Unique ID (removed)
Surname	Customer name (removed)

CreditScore	Credit score of customer
Geography	Country (France, Germany, Spain)
Gender	Male/Female
Age	Age of customer
Tenure	Years with bank
Balance	Account balance
NumOfProducts	Number of bank products
HasCrCard	Credit card (1 = Yes, 0 = No)
IsActiveMember	Active status
EstimatedSalary	Estimated salary
Exited	**Target (1 = churn, 0 = no churn)**

3. Mathematical Formulation of ANN

An Artificial Neural Network consists of layers of neurons.

- **Weighted Sum:**

$z=\sum i=1nwixi+bz=i=1\sum nwixi+b$

- **Activation Function (Sigmoid):**

$f(z)=11+e-zf(z)=1+e-z1$

- **Output Prediction:**

$y^=f(z)y^=f(z)$

- **Loss Function (Binary Cross-Entropy):**

$L=-(y\log(y^)+(1-y)\log(1-y^))L=-(y\log(y^)+(1-y)\log(1-y^))$

- **Weight Update (Gradient Descent):**

$w=w-\eta\partial L\partial ww=w-\eta\partial w\partial L$

4. Algorithm Limitations

- Requires **large datasets** for good performance
- Can **overfit** if not properly tuned
- Training can be **computationally expensive**
- Hard to interpret (black-box model)
- Sensitive to **feature scaling and preprocessing**

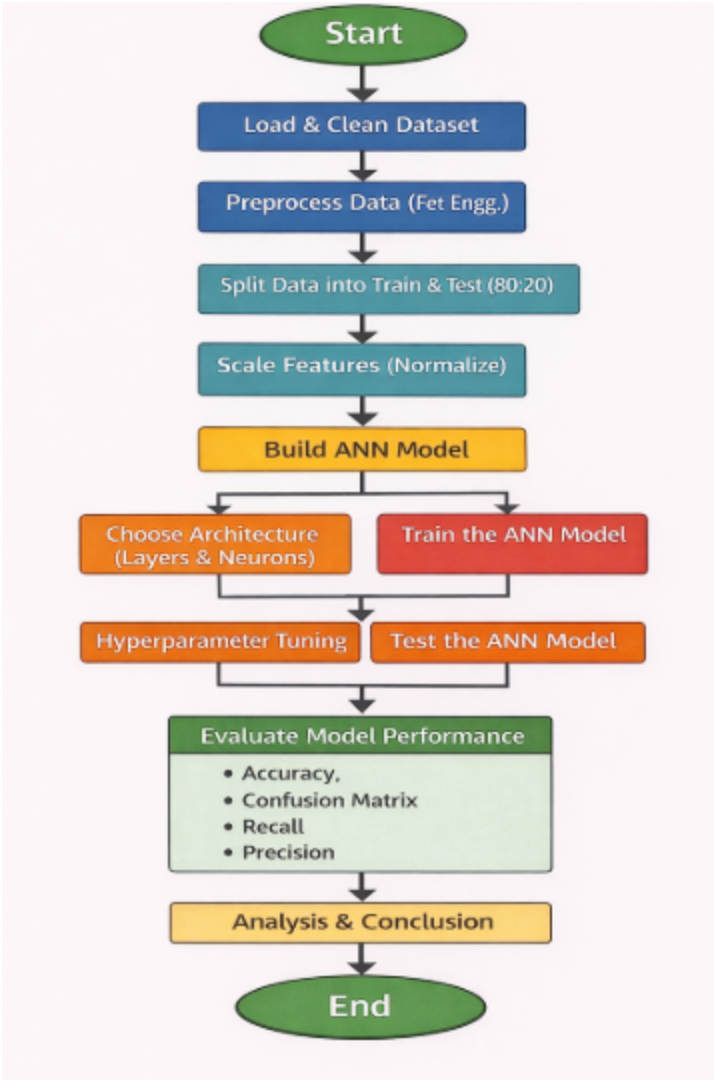
5. Methodology / Workflow

- **Steps Followed:**

1. **Data Loading**
   - Dataset imported into MATLAB
2. **Data Cleaning**
   - Removed irrelevant columns (RowNumber, CustomerId, Surname)
3. **Data Preprocessing**
   - Converted categorical variables (Gender, Geography) to numeric
   - Normalized feature values
4. **Train-Test Split**

    - 80% training, 20% testing
5. **Model Building**
    - Used patternnet (Feedforward ANN)
    - Hidden layer with neurons
6. **Model Training**
    - Trained using backpropagation
7. **Prediction**
    - Generated predictions on test data
8. **Evaluation**
    - Accuracy and confusion matrix

🔹 **Workflow Diagram**



6. Performance Analysis

🔹 **Evaluation Metrics Used:**

- **Accuracy**
- **Confusion Matrix**

🔹 **Accuracy Formula:**

*Accuracy=Correct PredictionsTotal Predictions*Accuracy=Total PredictionsCorrect Predictions

🔹 **Observations:**

- Model achieves approximately **80–85% accuracy**
- Good at predicting **non-churn customers**
- Slight imbalance in predicting churn cases

🔹 **Confusion Matrix Interpretation:**

- True Positives: Correct churn predictions
- True Negatives: Correct non-churn predictions
- False Positives/Negatives indicate errors

7. Hyperparameter Tuning

🔹 **Parameters Tuned:**

- Number of hidden neurons
- Number of hidden layers

🔹 **Experiments:**

Configuration	Accuracy
10 neurons	~80%
20 neurons	~83%
[10,5] layers	~85%

🔹 **Observations:**

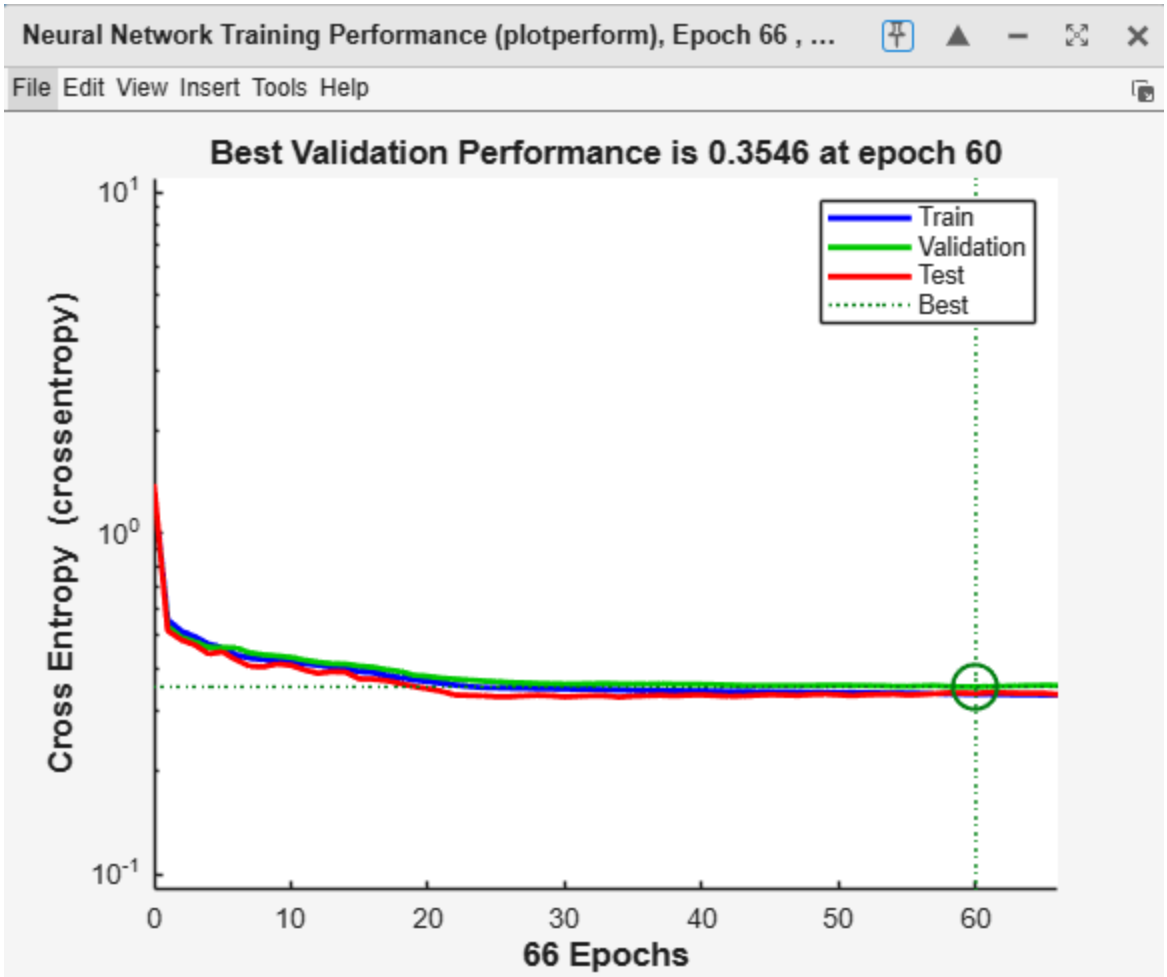
- Increasing neurons improved performance initially
- Too many neurons can cause **overfitting**
- Best performance achieved with **multi-layer network**

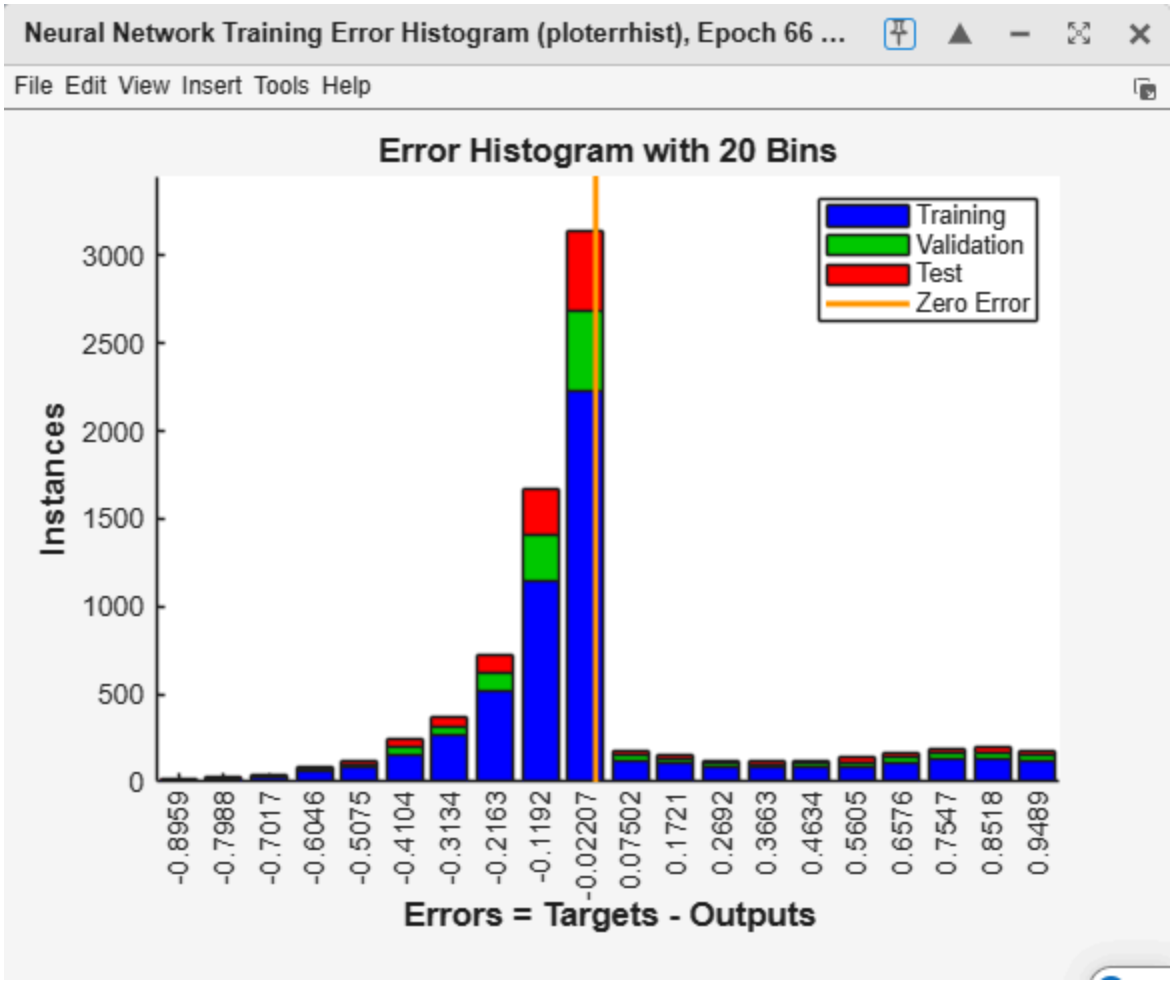
CODE

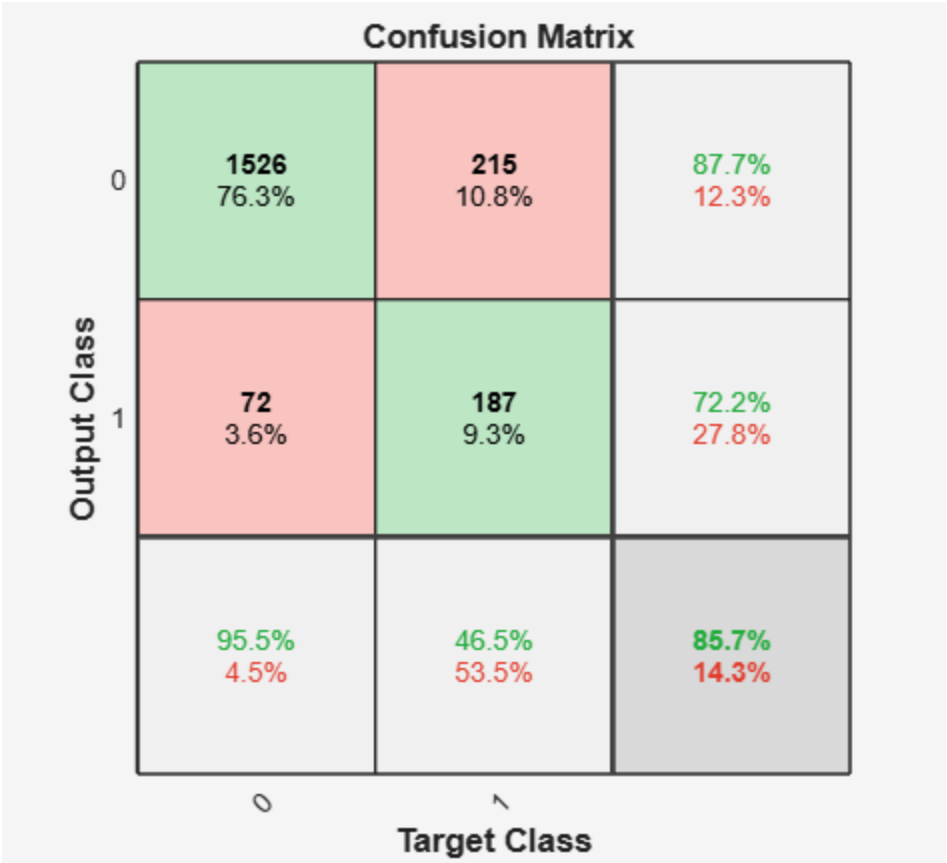
```
data = readtable('Churn_Modelling.csv');
head(data)
summary(data)
data.RowNumber = [];
data.CustomerId = [];
data.Surname = [];
data.Gender = grp2idx(data.Gender);     % Male/Female → 1,2
data.Geography = grp2idx(data.Geography); % France/Germany/Spain →
numbers
X = data(:,1:end-1);   % all except last column
Y = data.Exited;       % target column
X = table2array(X);
Y = data.Exited;
X = normalize(X);
cv = cvpartition(size(X,1),'HoldOut',0.2);
XTrain = X(training(cv),:);
YTrain = Y(training(cv),:);
XTest = X(test(cv),:);
YTest = Y(test(cv),:);
hiddenLayerSize = 10;
net = patternnet(hiddenLayerSize);
net = train(net, XTrain', YTrain');
YPred = net(XTest');
YPred = round(YPred);   % Convert probabilities to 0/1
accuracy = sum(YPred' == YTest) / length(YTest);
disp(['Accuracy: ', num2str(accuracy*100), '%']);
plotconfusion(YTest', YPred);
```

Output

```
Accuracy: 85.65%
```


Neural Network Training Performance (plotperform), Epoch 66 , ...
File Edit View Insert Tools Help
Best Validation Performance is 0.3546 at epoch 60
Train
Validation
Test
Best
Cross Entropy (crossentropy)
10^{1}
10^{0}
10^{-1}
0
10
20
30
40
50
60
66 Epochs


Neural Network Training Error Histogram (ploterrhist), Epoch 66 ...
File Edit View Insert Tools Help
Error Histogram with 20 Bins
Training
Validation
Test
Zero Error
Instances
3000
2500
2000
1500
1000
500
0
-0.8959
-0.7988
-0.7017
-0.6046
-0.5075
-0.4104
-0.3134
-0.2163
-0.1192
-0.02207
0.07502
0.1721
0.2692
0.3663
0.4634
0.5605
0.6576
0.7547
0.8518
0.9489
Errors = Targets - Outputs



Conclusion

- ANN successfully predicts customer churn
- Proper preprocessing significantly improves results
- Hyperparameter tuning enhances accuracy
- Model is effective but requires careful tuning